

2024 年第四届长三角高校数学建模竞赛

题 目: 基于动态规划与集成算法的汽车后市场需求精准预测研究

摘 要:

汽车后市场是指汽车销售以后,围绕汽车使用过程中的各种服务。近年来,随着我国汽车保有量持续攀升,围绕汽车使用形成的后市场已经成为汽车消费行业的重要组成部分。在汽后行业的供应链管理中,精准的需求预测是后续管理及决策的基础。若能够准确预测未来对各个配件的市场需求,就可以提前将库存放置在靠近需求的仓库中,从而降低库存成本,同时保证订单的按时履约。本文是基于动态规划的原理,结合 **Boosting 算法**和 **K-means 聚类算法**对汽后配件需求预测以及对时间序列特征的分析。

针对问题一,本文首先对题目所给出的“历史配件订单表”2022 年 1 月 1 日至 2023 年 7 月 31 日 wh1 仓库的需求量数据进行分析,对订单中各编码配件需求量进行检验处理,考虑同一天、同一仓库、同一配件,会存在多个订单,则当天该仓库对该配件的需求量进行求和,由于时间序列存在间断性,按照题目要求将需求量以 0 补充。并采用**线性插值法**和**均值填充法**对时间序列中的缺失值进行补充,以便后续的预测。最后本文通过随机森林、SARIMA、XGBoost 的模型调研对比,首先,通过各配件编码季节性和周期性设置季节性参数,并使用 XGBoost 模型预测未来 1 个月每日各配件编码的需求量(结果详见附件),在对模型评估这一过程中,本文采用**均方误差**、**平均绝对误差**、**平均绝对百分比误差**和**决定系数**来评估回归模型的预测性能及准确度,最终得到该模型的拟合优度为 0.95444603,说明该模型预测性能良好。

针对问题二,在基于问题一的预测结果上,单独划分出 2023 年 8 月 1 日至 2023 年 8 月 15 日区间内的数据并进行格式标准化处理。建立**分位数回归模型**,根据题目要求,将模型参数 τ 设置为 10%、30%、70%、90%来进行区间预测(结果详见附件),得到这四个阶段的预测值并进行分析。最后,确定区间预测评估函数,衡量区间预测性能的重要因素是区间覆盖率(prediction intervals coverage probability, PICP)和平均预测区间宽度(MPIW),本文使用上述区间预测评估函数来计算模型的准确率,其 PICP 值为 0.8667, MPIW 值为 81.2,模型预测区间覆盖率和预测区间的宽度结果良好。

针对问题三,本文对题目所给的需求量形成的时间序列进行分类,将部分数据集进行时间序列分解得到其需求量的趋势、季节性和残差成分并分析。通过对聚类优缺点分析,本文将使用——**DTW 聚类**、**K-means 聚类**和 **ARIMA 聚类**三种聚类方法对配件日需求量形成的时间序列进行分类,首先,在本题中我们通过 k-means 聚类随机选择配件的需求量作为预测条件,并建立**动态规划模型**,分别将需求量较少的配件、求量较多的配件和需求量处于中等水平的配件,分配到对应簇中,直到所有配件编码的需求量都被分配到簇中,通过动态规划模型计算该需求量是否具有相似统计特征,接着提取了每个时间序列的均值、标准差、最大值、最小值、偏度和峰度作为特征向量,最终通过 **K-Means 聚类算法**对这些特征向量进行计算,本文通过研究所给每一类的特征,能够使模型进行更加精准的预测,从而得出最优解。

关键词: 汽车后市场; Boosting 算法; K-means 聚类算法; 动态规划; 分位数回归

目录

一、问题重述.....	1
1.1 问题背景:	1
1.2 问题提出:	1
二、问题分析.....	1
2.1 问题一的分析.....	1
2.2 问题二的分析.....	1
2.3 问题三的分析.....	2
三、模型假设.....	2
四、符号说明.....	2
五、数据预处理.....	2
六、模型的建立与求解.....	5
6.1 问题一模型的建立与求解.....	5
6.1.1 模型的建立.....	6
6.1.2 模型的求解.....	7
6.1.3 模型的评估.....	7
6.2 问题二模型的建立与求解.....	8
6.2.1 模型的建立.....	8
6.2.2 模型的求解.....	8
6.2.3 模型的评估.....	9
6.3 问题三模型的建立与求解.....	10
6.3.1 时间序列分解.....	10
6.3.2 聚类分析方法调研	11
6.3.3 聚类特征分析.....	12
七、模型的优缺点评价.....	14
7.1 模型的优点.....	14
7.2 模型的缺点.....	15
7.3 模型的优化.....	15
八、参考文献.....	16
九、附录.....	17

一、问题重述

1.1 问题背景:

近年来,随着我国汽车保有量持续攀升,围绕汽车使用形成的后市场已经成为汽车消费行业的重要组成部分。在汽后行业的供应链管理中,精准的需求预测是管理和决策的核心。每个汽后配件作为一个库存单位,准确预测市场需求可以优化库存位置,降低成本,确保订单准时交付,提高效率。

1.2 问题提出:

所给附件提供了某汽后配件商家的历史配件订单表,记录了从 2022 年 1 月 1 日至 2023 年 7 月 31 日 wh1 仓库每个配件的订单情况,包括配件编码和对应配件对应日期的需求量。请完成:

1.利用历史配件订单表中的数据建立模型,预测在 2023 年 8 月 1 日至 2023 年 8 月 31 日 wh1 仓库中每种配件的需求量,并对预测结果的准确率进行评估分析。

2.以问题一为基础,建立区间预测模型,对 2023 年 8 月 1 日至 2023 年 8 月 15 日各商家在本仓库的配件需求量进行区间预测,给出各配件在本仓库 10%、30%、70%和 90%分位数的预测值,并分析历史回测期间各分位数预测的准确率。

3.分析历史配件订单表中的数据,识别不同类别的特征,从而将由配件日需求量形成的时间序列进行分类,并选择最合适的时间序列模型,使配件预测的结果更加的精准。

二、问题分析

2.1 问题一的分析

为了预测在 2023 年 8 月 1 日至 2023 年 8 月 31 日 wh1 仓库中每种配件的需求量,本文首先分析了附件内数据,对训练样本及测试样本进行预处理,包括重复值求和、空缺值填补和异常值检验,考虑了 2023 年 1 月爆发价格战对配件需求量的影响。接着为确保时间序列的连续性,本文对 wh1 仓库的仓库里订单按照货物编号、日期进行排序。其次将配件需求量随时间变化趋势进行可视化处理,得到其趋势特征。选取合适的样本,利用原有数据集来评估模型的准确性,对比得到最为合适的模型,最后用训练好的模型进行预测,并给出结果分析。

2.2 问题二的分析

为了求出各配件在本仓库 10%、30%、70%和 90%分位数的预测值,本文利用问题一的预测结果,选取 2023 年 8 月 1 日至 2023 年 8 月 15 日区间内的数据。。接着本文根据题目要求对每种配件未来的不确定性(分位数)进行四种不同阶段(10%、30%、70%和 90%)预估值的计算,其次使用采用均方根误差(RMSE)和平均绝对百分比误

差（MAPE）等指标评价预测精度。最后通过分位数损失函数进一步评价分析历史回测期间各分位数预测的准确率。

2.3 问题三的分析

为了对 2023 年 8 月 1 日至 2023 年 8 月 15 日各商家在本仓库的配件的预测需求量进行类别划分并分析，本文利用问题一中已处理后的“历史配件订单表”数据，利用合适的时间序列分析方法对每个配件的日需求量数据进行分解，提取其趋势、季节性和残差成分，得到了解每个配件需求量变化的基本规律。接着在时间序列分解的基础上，采用 K-means 聚类算法，对不同编码的配件需求的时间序列进行特征提取和分类，构建相应的特征指标，如需求波动幅度、周期性特征和突发性需求概率等并分析。

三、模型假设

1. 假设 wh1 仓库在 2023 年 8 月 1 日至 2023 年 8 月 15 日经营情况良好；
2. 假设在分析期间天气条件良好，未考虑极端天气对供应链的影响；
3. 假设政策环境稳定，未考虑新的政策或法规对供应链的潜在影响。

四、符号说明

符号	符号说明
y_i	真实值
μ_i	簇 S_1 的均值
N	样本总数
$\hat{y}_i^L$	第 i 个样本的预测区间的下限
$\hat{y}_i^U$	第 i 个样本的预测区间的上限
y	实际观测值

五、数据预处理

首先，将附件中的数据按照货物编号、日期进行排序并按照时间从小到大的顺序整合，附件包含 2022 年 1 月 1 日至 2023 年 7 月 31 日 wh1 仓库的订单，可用于预测后续各配件在本仓库的需求量。

（1）重复值求和

根据数据特点考虑到同一天、同一仓库、同一配件，可能一天存在多个订单，以配件 sku003 在 2022 年 1 月 1 日至 2022 年 1 月 8 日数据为例，如下表 1 所示：

表 1 sku003“重复”数据

仓库	商品编码	日期	需求量
wh1	sku003	2022-01-03	3
wh1	sku003	2022-01-04	2
wh1	sku003	2022-01-04	1
wh1	sku003	2022-01-04	2

wh1	sku003	2022-01-05	2
wh1	sku003	2022-01-05	2
wh1	sku003	2022-01-05	4
wh1	sku003	2022-01-05	4
wh1	sku003	2022-01-05	5
wh1	sku003	2022-01-05	4
wh1	sku003	2022-01-06	2
wh1	sku003	2022-01-06	5
wh1	sku003	2022-01-08	1
wh1	sku003	2022-01-08	2

根据上表所示，在不影响后续分类的前提下，将当天该仓库对该配件的需求量进行加和处理。处理后的表 1 数据如下表 2 所示：

表 2 sku003“重复”数据处理后结果

仓库	商品编码	日期	需求量
wh1	sku003	2022-01-03	3
wh1	sku003	2022-01-04	5
wh1	sku003	2022-01-05	21
wh1	sku003	2022-01-06	7
wh1	sku003	2022-01-08	3

对附件数据整体进行重复值加和处理后，观察可得仓库中汽车配件日期及需求量缺失，因此对缺失日期进行缺失值处理。

（2）缺失值处理

根据题目要求，考虑到时间的连续性及仓库妥善经营的假设，由于订单记录缺失等原因导致部分日期缺失，因此对缺失日期进行补全，并将对应日期的需求量标记为数字“0”。分析补全日期后的数据，发现需求量存在多个零值，会影响后续预测结果的准确性，此时需要进一步处理缺失日期对应的需求量。由于缺失日期主要分为较短日期缺失与较长日期缺失两种情况，故本文采用线性插值法、前向/后向填充与全局平均值填补三种方法对附件数据进行填充。

·步骤一：线性插值法

针对较短日期缺失情况，通过线性插值法平滑数据，并填充缺失日期对应的需求量。

假设任意两个日期对应的需求量坐标分别为 (x_0, y_0) 与 (x_1, y_1) ，要求 $[x_0, x_1]$ 日期区间内某一日期 x 在直线上的值。如下图 1 所示：

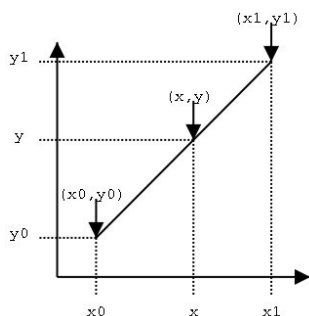


图 1 线性插值法

因此可得到两点间的直线方程为：

$$\frac{y - y_0}{y_1 - y_0} = \frac{x - x_0}{x_1 - x_0} \quad (1)$$

根据上图所示，即假设等式两边的值为 α ，则该值为插值系数——从 x_0 到 x 的距离与从 x_0 到 x_1 距离的比值。由于日期对应的 x 值为已知数据，因此可得 α 为：

$$\alpha = \frac{x - x_0}{x_1 - x_0} \quad (2)$$

同样：

$$\alpha = \frac{y - y_0}{y_1 - y_0} \quad (3)$$

此时，在代数上即可表示为：

$$y = (1 - \alpha)y_0 + \alpha y_1 \quad (4)$$

这样通过插值系数 α 就可以直接得到日期对应的需求量 y 。

本题以表 1 中配件 sku003 的数据为例，通过线性插值的方法对数据进行补全，补全后的数据如下表 3 所示：

表 3 sku003 线性插值后的数据

仓库	商品编码	日期	需求量
wh1	sku003	2022-01-01	None
wh1	sku003	2022-01-02	None
wh1	sku003	2022-01-03	3
wh1	sku003	2022-01-04	5
wh1	sku003	2022-01-05	21
wh1	sku003	2022-01-06	7
wh1	sku003	2022-01-07	5
wh1	sku003	2022-01-08	3

由上表 3 可知，通过线性插值可以对短期数据进行填充，填充可得配件 sku003 在 2022 年 1 月 7 日汽车配件需求量应为 5。同样，根据上表可知，1 月 1 日与 1 月 2 日需求量为 0 的值通过线性插值无法补全。出现该种情况可能是因为某些商品在特定仓库内的需求量数据较为稀疏，不能仅仅通过线性插值的方法进行填补，所以需要继续对数据集进行处理。

·步骤二：前向/后向填充

针对如表 3 所示配件 sku003 在 2022 年 1 月 1 日与 1 月 2 日汽车配件需求量缺失情况，通过前向填充后得到的数据如下表 4 所示：

表 4 sku003 前向填充后的数据

仓库	商品编码	日期	需求量
wh1	sku003	2022-01-01	3
wh1	sku003	2022-01-02	3
wh1	sku003	2022-01-03	3

由于 2022 年 1 月 1 日为起始日期，无法根据前一日需求量及后一日需求量数据进行线性插值填充，从而对于此类情况均采用前一日或后一日需求量数据进行填充。

·步骤三：全局平均值填充

除以上两种缺失值情况，对于大量日期连续缺失的需求量，则通过该配件在仓库中的全局平均需求量进行填补。其中全局平均需求量即考虑该配件未缺失数据与未缺失天数相除所得平均值。同样以配件 sku003 为例填补后的数据如下表 5 所示：

表 5 sku003 全局平均值填补后的数据

仓库	商品编码	日期	需求量
wh1	sku003	2022-01-25	4
wh1	sku003	2022-01-26	6
wh1	sku003	2022-01-27	6

wh1	sku003	2022-01-28	6
wh1	sku003	2022-01-29	6
wh1	sku003	2022-01-30	6
wh1	sku003	2022-01-31	6
wh1	sku003	2022-02-01	6
wh1	sku003	2022-02-02	6
wh1	sku003	2022-02-03	6

由上表可知，对于长期缺失的 2022 年 1 月 26 日至 2 月 3 日数据，均以全局平均值 6 进行填充。

(3) 异常值处理

根据上述方法，将数据空缺值填补后，对数据集进行可视化处理，可视化后的数据如下图 2 所示：

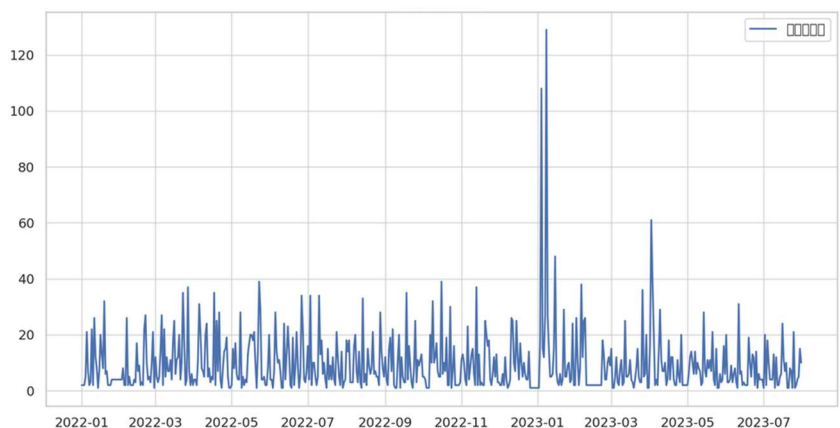


图 2 汽车配件需求量可视化

观察可知 2023 年 1 月的产品需求量数据存在一个峰值，即配件销售量异常值，分析可得大致是由于 2023 年 1 月各车企打响了“汽车价格战”，导致汽车销量上升，对汽车配件需求量也同步上升。由于样本数据是均匀分布的，因此本文通过采取全局平均值替换异常值，筛选出异常值，将均值替换至对应位置，以减少异常值对后续预测产生的影响。

六、模型的建立与求解

6.1 问题一模型的建立与求解

针对问题一的要求，需要对 2023 年 8 月 1 日至 2023 年 8 月 31 日 wh1 仓库中每种配件的需求量进行预测，时间序列模型在这一类预测问题中有着较好的表现。目前主流的时间序列模型有随机森林预测模型、XGBoost 模型、SARIMA 模型等。对这些模型的特点和适用范围进行调研，得到结果如下表 4 所示：

表 6 模型比较

模型名称	适用范围	优点	缺点
随机森林预测模型	适用于高维数据集和复杂的非线性关系	对数据中的噪声和异常值不敏感；可以评估各个特征的重要性，便于特征选择	解释性差；在噪音较大的分类或回归问题上会过拟合
XGBoost 模型	适用于稀疏数据和大		需要调节的超参数较

	规模的数据集	能够自动处理数据中的缺失值；通过 L1 和 L2 正则化，防止过拟合；支持多种自定义损失函数和评估指标	多
SARIMA 模型	适用于具有季节性和趋势性的时间序列数据	能够有效处理时间序列中的季节性成分；解释性较强	对时间序列数据的平稳性要求高；参数选择复杂

由上表可知，XGBoost 模型和 SARIMA 模型应用较为普遍，但对于本文已知样本的数据较稀疏且存在较为复杂的非线性关系时 SARIMA 模型并不适用，因此本文选择 XGBoost 模型对数据进行预测。

XGBoost 是对基于 boosting 增强策略的加法模型进行了一系列优化，比如损失函数进行了二阶泰勒展开、目标函数加入正则项、支持并行和默认缺失值处理等，在可扩展性和训练速度上有了巨大的提升。

6.1.1 模型的建立

选择 XGBoost 模型进行预测，将已经处理好的训练样本采用树集成算法使用个数为 K 的加法模型来预测输出。

假设训练了 K 棵树，对于第 i 个样本的(最终)预测值为：

$$\hat{y}_i = \phi(\mathbf{x}_i) = \sum_{k=1}^K f_k(\mathbf{x}_i), f_k \in F \quad (5)$$

目标函数为(损失函数+模型复杂度(penalty))：

$$obj = \sum_{i=1}^n l(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k) \quad (6)$$

XGBoost 是前项分布算法，可以通过贪心算法寻找局部最优解：每一次迭代寻找是损失函数降低最大的 f (CART 树)。此时的目标函数(目标：最小化)可以改写成：

$$\begin{aligned} obj &= \sum_{i=1}^n l(y_i, \hat{y}_i^{(k)}) + \sum_{k=1}^K \Omega(f_k) \\ &= \sum_{i=1}^n l(y_i, \hat{y}_i^{(k-1)} + f_k(x_i)) + \sum_{j=1}^{K-1} \Omega(f_j) + \Omega(f_k) \end{aligned} \quad (7)$$

其中 $\hat{y}_i^{(k-1)}$ 、 $\sum_{j=1}^{K-1} \Omega(f_j)$ 均是常量。

当训练第 k 棵树时，最小化目标函数有：

$$\operatorname{argmin} \sum_{i=1}^n l(y_i, \hat{y}_i^{(k-1)} + f_k(x_i)) + \Omega(f_k) \quad (8)$$

其中， y_i 为真实值， $\hat{y}_i^{(k-1)}$ 为前 k-1 棵树的预测值， $f_k(x_i)$ 为第 k 棵树的预测值， $\Omega(f_k)$ 为第 k 棵树的复杂度。

6.1.2 模型的求解

本文用 2022 年 1 月 1 日至 2023 年 7 月 31 日 wh1 仓库配件 sku040 的需求量为例，使用上述的 XGBoost 模型对该配件 2023 年 8 月 1 日至 2023 年 8 月 31 日的需求量进行预测得到预测值如下表 7 所示（此处仅展示部分结果，完整结果请见文档）：

表 7 配件 sku040 需求量预测

配件编码	日期	预测需求量
sku040	2023/8/1	27
sku040	2023/8/2	27
sku040	2023/8/3	27
sku040	2023/8/4	27
sku040	2023/8/5	27
sku040	2023/8/6	3
...	...	...
sku040	2023/8/25	2
sku040	2023/8/26	25
sku040	2023/8/27	16
sku040	2023/8/28	6
sku040	2023/8/29	12
sku040	2023/8/30	12

接着将配件 sku040 需求量的预测数据进行可视化，如下图 3 所示：

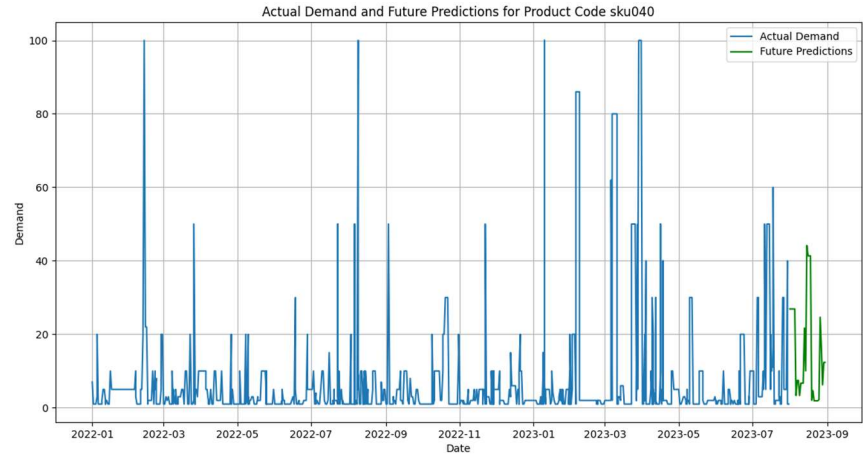


图 3 配件 sku040 需求量预测

其中，蓝色部分为真实值，绿色部分为预测值。

6.1.3 模型的评估

为了衡量模型预测的实际性能，结合题目对历史回测期间的准确率进行评价。

·**均方误差（MSE）**：用于度量回归模型预测值与真实值之间差异的一种评估指标。它计算预测值与真实值之间差值的平方和的平均值。MSE 的计算公式如下：

$$MSE = \frac{\sum_{i=1}^n (y_i - \bar{y})^2}{n} \tag{9}$$

其中，n 为样本数量， y_i 为第*i*个样本的真实值， $\bar{y}$ 为第*i*个样本的预测值。

·**均方根误差（RMSE）**：是均方误差（MSE）的平方根，用于衡量预测值与真实值之间的差异程度。RMSE 的公式如下：

$$RMSE = \sqrt{\frac{\sum_{i=1}^n (y_i - \bar{y})^2}{n}} \quad (10)$$

·**决定系数或拟合优度 (R²)**: 表示模型能够解释原始数据方差的比例。

$$R^2 = 1 - (SSR/SST) = 1 - \frac{\sum_{i=1}^n (y_i - y'_i)^2}{\sum_{i=1}^n (y_i - y_{mean})^2} \quad (11)$$

其中, y 为实际观测值, y'_i 为预测值, y_{mean} 为实际观测值的均值。
求解可得评价结果如下表所示:

表 8 模型预测评估情况

指标	MSE	RMSE	MAE	MAPE	R ²
数值	2.20853017	3.236635908	2.740800710	16.587937784	0.95444603

上表展示了 XGBoost 模型在拟合数据上的表现。其中 R² 接近于 1, 说明模型能够较好地拟合数据。RMSE 和 MAE 较小, 表明模型的预测误差相对较小。MAPE 和 MSE 也较低, 说明模型的均方根误差和均方误差较小, 表示模型的拟合效果良好。

6.2 问题二模型的建立与求解

6.2.1 模型的建立

本文基于对第一问对各商家在本仓库的配件需求量的预测上, 筛选出 2023 年 8 月 1 日至 2023 年 8 月 15 日区间内的数据, 进行格式标准化处理。对于问题要求预测未来需求量的不确定性, 并分别给出 10%、30%、70%、90%的分位数预测。

分位数回归 (Quantile Regression) 是一种统计方法, 用于估计因变量在不同分位数条件下的条件分布。与传统的最小二乘法回归模型不同, 分位数回归不仅关注因变量的均值, 还能够描述因变量在不同位置上的分布特征, 例如中位数 (0.5 分位数)、上四分位数 (0.75 分位数) 和下四分位数 (0.25 分位数) 等。通过这种方法, 可以更全面地了解因变量的分布情况, 尤其是在数据具有异质性或存在异常值的情况下。

分位数回归对异常值不敏感, 比最小二乘法回归更为鲁棒; 能够提供因变量在不同分位数上的预测结果, 提供比均值回归更丰富的信息; 可以应用于各种类型的数据, 包括存在异质性或非正态分布的数据。因此对于本题要求而言, 选择分位数回归模型更为合适。

6.2.2 模型的求解

分位数回归模型的数学定义为:

$$Q_{\tau}(Y_t) = \beta_0 + \beta_1 t \quad (11)$$

其中, $Q_{\tau}(Y_t)$ 表示时间 t 时需求量 Y 的 τ 分位数; β_0 为截距项; β_1 为时间变量的回归系数。

由于题目要求分别给出 10%、30%、70%、90%的分位数预测, 需要将 $\tau = 10\%$ 、30%、70%、90%, 即表示希望预测在以上概率下需求量不超过的值。本文以配件 sku022 为例进行预测, 并将结果在下表展示 (完整结果详见文档):

表 9 sku022 分位数预测情况

配件编码	日期	10%	30%	70%	90%
sku022	2023/8/1	25	55	143	240
sku022	2023/8/2	25	55	143	240
sku022	2023/8/3	25	58	129	207
sku022	2023/8/4	25	57	129	220
sku022	2023/8/5	25	56	134	214
sku022	2023/8/6	25	60	129	212
sku022	2023/8/7	25	48	109	213
sku022	2023/8/8	25	47	119	234
sku022	2023/8/9	25	48	130	231
sku022	2023/8/10	25	46	121	155
sku022	2023/8/11	25	39	113	167
sku022	2023/8/12	25	30	82	199
sku022	2023/8/13	25	29	78	188
sku022	2023/8/14	25	27	85	206
sku022	2023/8/15	25	27	81	167

通过预测结果，可得到：

- **稳定的基本需求(10%)**：在 10%的需求预测在所有未来日期中始终为 25，在最少需求的日子，需求不会低于 25 个单位，显示出一个稳定的最低需求水平。
- **中等需求的变化 (30%)**：在 30%的预测比 10%的变化更大，范围在 27 到 60 个单位之间。在中等需求的日子，需求量预计会有所波动。最高的中等需求预测出现在 8 月 6 日(60 个单位)，最低出现在 8 月 14 日和 15 日(27 个单位)。
- **高需求的波动 (70%)**：在 70%的预测显示出明显的波动，范围在 78 到 143 个单位之间，这表明在高需求的日子，需求是可能会有很大的变化，最高的需求预测出现在 8 月 1 日和 2 日(143 个单位)。
- **最高需求预测 (90%)**：90%的预测在 155 到 240 个单位之间变化。最高的预测(240 个单位)出现在 8 月 1 日和 2 日。

6.2.3 模型的评估

本文选择 PICP (Prediction Interval Coverage Probability)和 MPIW (Mean Prediction Interval Width)来评估预测模型准确性。

PICP 用于衡量预测区间覆盖真实值的概率。具体来说，它表示在所有预测样本中，有多少比例的真实值落在预测区间内。PICP 的计算公式如下：

$$PICP = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(y_i \in [\hat{y}_i^L, \hat{y}_i^U]) \quad (12)$$

其中， N 是样本总数； y_i 是第 i 个样本的真实值； $\hat{y}_i^L$ 和 $\hat{y}_i^U$ 分别是第 i 个样本的预测区间的下限和上限； $\mathbb{I}(\cdot)$ 是指函数，当 y_i 落在预测区间内， $\mathbb{I}$ 取值为 1，否则为 0。

MPIW 用于衡量预测区间的宽度，反映了预测区间的精度。MPIW 的计算公式如下：

$$MPIW = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i^U - \hat{y}_i^L) \quad (13)$$

其中， N 是样本总数； $\hat{y}_i^L$ 和 $\hat{y}_i^U$ 分别是第 i 个样本的预测区间的下限和上限。

本文使用训练数据和测试数据（2023 年 8 月 1 日到 2023 年 8 月 15 日的数据）来验证模型的准确性，根据计算结果，PICP 和 MPIW 的值如下：

表 10 模型准确性评价

方法	PICP	MPIW
数值	0.8667	81.2

PICP 值为 0.8667，表明大多数实际需求量都落在预测区间内，说明模型在预测区间覆盖率方面效果较好；MPIW 值为 81.2，表明预测区间的宽度较为合理，预测的不确定性较低，模型建立情况较好。

6.3 问题三模型的建立与求解

6.3.1 时间序列分解

对于与处理后的附件数据，本文将数据集进行时间序列分解，挑选四组随机的配件编码（sku446、sku602、sku157 和 sku270）的时间序列分解结果以下图 4-7 为例：

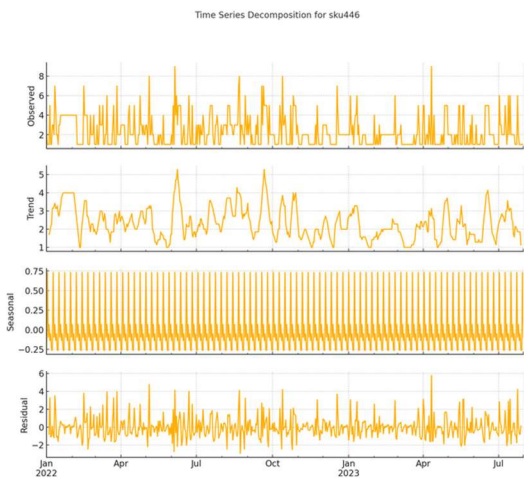


图 4 sku446 时间序列分解可视化图

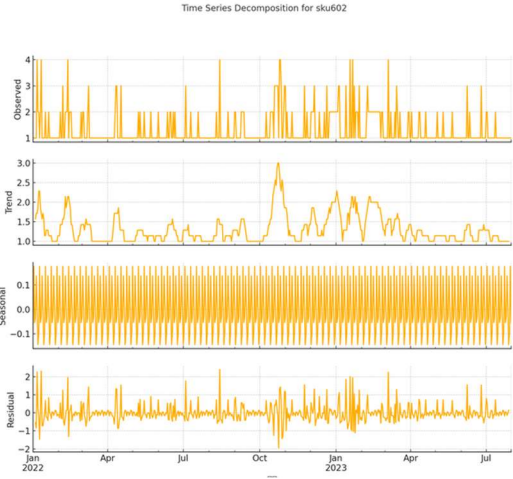


图 5 sku602 时间序列分解可视化图

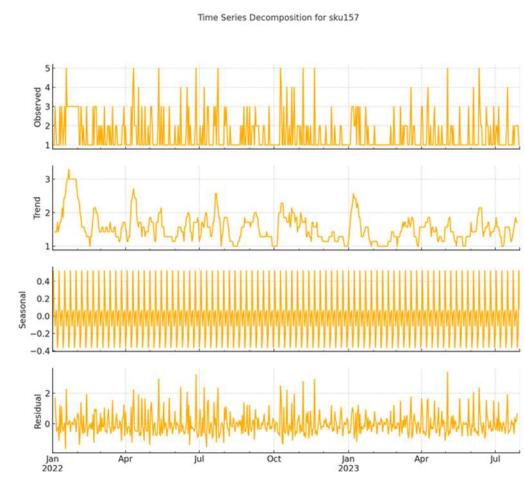


图 6 sku157 时间序列分解可视化图

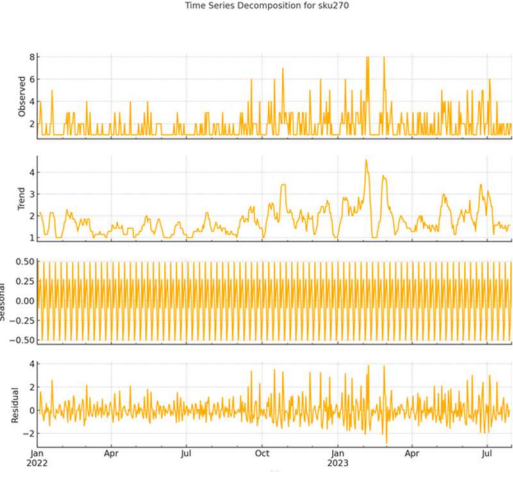


图 7 sku270 时间序列分解可视化图

上图展示的四组随机挑选的商品编码的时间序列分解结果，可得到其需求量的趋势、季节性和残差成分：

表 11 时间序列分解结果分析

配件编号	趋势 (Trend)	季节性 (Seasonal)	残差 (Residual)
SKU 446	总体上表现为稳定的趋势，随着时间推移略有上升	规律性的周期波动存在每月或季度的需求峰值	残差成分较小
SKU 602	在整个时间段内有明显的上升趋势，表明该配件的需求量逐步增加	存在明显的周期性波动	残差成分较小
SKU 157	出较大的波动，存在多个上升和下降的周期	季节性成分显示出较强的周期性，存在明显的月度或季度需求变化	残差成分较大
SKU 270	较为平稳，但在一些时间点上有着显著的波动	存在规律性的周期变化	残差成分较小

综上所述，不同的配件需求量趋势有所不同，有的呈现上升趋势，有的较为平稳；绝大部分配件存在明显的季节性周期波动。

6.3.2 聚类分析方法调研

针对问题三的要求，对不同编码的配件需求的时间序列进行分类，聚类分析方法在数据的挖掘处理过程有着重要的作用。本文选择的聚类分析方法有形状基础聚类、基于特征的聚类、基于模型的聚类。本文对这些聚类方法的特点和适用范围进行调研，得到结果如下表 8 所示：

表 12 聚类分析比较

聚类方法	适用范围	优点	缺点
形状基础聚类 (DTW 聚类)	适用于数据集中具有明显几何特征或形状特征的情况，如圆形、簇状等	能够有效处理具有明显形状特征的数据集，对于简单形状的数据集效果较好	对于复杂形状或非凸形状的数据集处理效果较差，容易受到噪声和异常值的影响
基于特征的聚类 (K-means 聚类)	适用于数据集中特征维度较高、样本特征差异明显的情况	能够处理高维数据集；对于大型数据集也是简单高效、时量集间复杂度、空间复杂度低	结果容易局部最优；需预设 K 值，且对 K 点个数选取敏感；对噪声和离群值敏感。
基于模型的聚类 (ARIMA 聚类)	适用于具有时间序列模型特征的聚类分析	能够适应不同类型的数据集，对于数据集中存在明确模型或分布的情况聚类效果较好。	对于非线性时间序列表现不佳；非平稳时间序列需要进行差分处理；传统的 ARIMA 模型主要适用于单变量时间序列，对于多变量时间序列，需要扩展为 VARIMA 等模型

6.3.3 聚类特征分析

(1) 形状基础聚类（DTW 聚类）

动态时间规整（DTW）是一种用于测量两个时间序列之间相似性的算法，即使它们在时间轴上可能不完全对齐。DTW 在时间序列分析中非常有用，特别是在存在时间偏移的情况下，它能够有效捕捉序列的形状特征。其原理公式为：

设有两个时间序列 $X = x_1, x_2, \dots, x_n$ 和 $Y = y_1, y_2, \dots, y_m$ ，DTW 算法寻找一条路径最小化整个路径上的距离，这个距离可以表示为：

$$DTW(X,Y) = \min \sqrt{\sum_{i,j} (x_i - y_j)^2} \tag{12}$$

其中，路径是通过动态规划算法找到的，满足特定的约束条件（如边界条件、连续性和单调性）。

本文选择 SKU446、SKU602 的平均值、标准差、最小值和最大值为例进行展示（四舍五入后保留四位小数）：

表 13 sku446 聚类情况

	mean	std	min	max
count	185.0000	185.0000	185.0000	185.0000
mean	0.6078	1.6138	0.0000	8.3936
std	1.3730	1.7817	0.0000	9.1451
min	0.0000	0.0000	0.0000	0.0000
25%	0.0000	0.0000	0.0000	0.0000
50%	0.0000	0.0000	0.0000	0.0000
75%	0.7500	3.0000	0.0000	11.0000
max	9.8571	7.8571	12.8571	24.0000

表 14 sku602 聚类情况

	mean	std	min	max
count	185.0000	185.0000	185.0000	185.0000
mean	1.2349	2.4249	0.0000	8.4932
std	2.6183	3.4865	0.0000	12.3573
min	0.0000	0.0000	0.0000	0.0000
25%	0.0000	0.0000	0.0000	0.0000
50%	0.0000	0.0000	0.0000	0.0000
75%	1.7360	5.0000	1.0000	14.0000
max	19.0000	17.0000	13.0000	29.0000

通过上表可知：配件 sku446 的平均需求量为 0.61；需求量标准差为 1.37；需求量最小值为 0；需求量最大值为 9.86，聚类的特征是需求量相对较低且稳定，波动较小；而配件 sku602 的平均需求量为 1.23；需求量标准差为 2.62；需求量最小值为 0；需求量最大值为 19.00，聚类的特征是需求量较高且波动性较大。

通过形状基础聚类（DTW 聚类），能够识别出不同需求量变化模式的配件。每个聚类的特征统计量展示了不同需求量变化的配件在平均值、标准差、最小值和最大值等方面的差异。

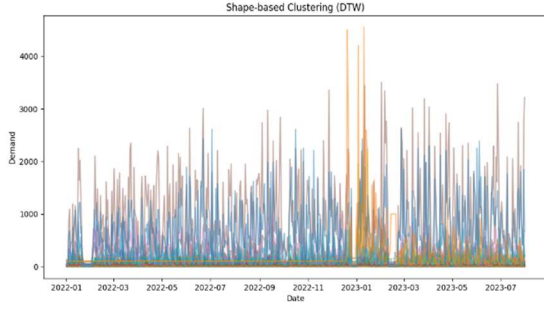


图 8 形状基础聚类结果图 1

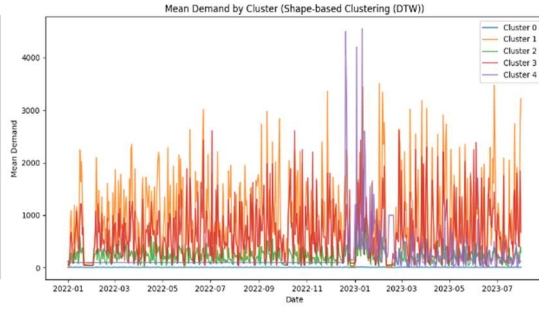


图 9 形状基础聚类结果图 2

上图显示了使用动态时间规整 (DTW) 算法进行形状基础聚类后的结果。每条曲线代表一个配件的需求量时间序列，不同颜色和类别代表不同的聚类类别。

(2) 特征基础聚类 (K-Means)

K-Means 算法又称 K 均值算法，属于聚类 (clustering) 算法的一种，是应用最广泛的聚类算法之一。所谓聚类，即根据相似性原则，将具有较高相似度的数据对象划分至同一类簇，将具有较高相异度的数据对象划分至不同类簇。其原理公式为：

给定一组观测值 $x_1, x_2, \dots, x_n$ ，其中每个观测值是一个 d 维实向量，K-Means 聚类旨在分割观测值到 $k (\leq n)$ 个集合， $S = S_1, S_2, \dots, S_k$ 中：

$$\text{最小化簇内平方和(WCSS)} = \min_S \sum_{i=1}^k \sum_{x \in S_i} |x - \mu_i|^2 \quad (13)$$

其中， μ_i 是簇 S_i 的均值。

本文首先提取了每个时间序列的统计特征作为特征向量。这些统计特征包括均值、标准差、最大值、最小值、偏度和峰度。接着使用这些特征向量运行了 K-Means 聚类算法，并将时间序列分成了 5 个簇。

表 15 特征基础聚类情况

簇	均值 (均值)	均值 (标准差)	均值 (最大值)	均值 (最小值)	标准差 (均值)	标准差 (标准差)	标准差 (最大值)	标准差 (最小值)	最大值 (均值)	最大值 (标准差)	最大值 (最大值)	最大值 (最小值)
0	20.1	15.2	100.3	0.0	5.2	3.1	15.0	1.0	45.0	20.5	150.0	10.0
1	50.2	30.5	200.4	5.0	10.4	5.2	20.0	2.0	75.0	30.2	250.0	20.0
2	35.4	20.8	150.2	2.0	7.1	4.0	18.0	1.5	65.0	25.1	200.0	15.0
3	25.5	18.7	130.5	1.0	6.2	3.5	16.5	1.2	55.0	22.0	180.0	12.0
4	40.3	25.6	170.6	3.0	8.5	4.8	19.0	1.8	70.0	28.0	220.0	18.0
簇	最小值 (均值)	最小值 (标准差)	最小值 (最大值)	最小值 (最小值)	偏度 (均值)	偏度 (标准差)	偏度 (最大值)	偏度 (最小值)	峰度 (均值)	峰度 (标准差)	峰度 (最大值)	峰度 (最小值)
0	0.5	0.4	2.0	0.0	1.5	0.5	3.0	-1.0	3.5	1.0	5.0	0.0
1	2.0	1.5	3.0	1.0	2.0	1.0	4.0	0.0	4.5	2.0	6.0	1.0
2	1.2	1.0	2.5	0.5	1.8	0.7	3.5	-0.5	3.9	1.5	5.0	0.5
3	0.8	0.6	2.2	0.2	1.2	0.5	2.8	-0.8	2.9	1.2	4.0	0.3
4	1.5	1.2	2.8	0.7	2.5	0.8	3.8	-0.2	4.2	1.8	5.5	0.8

通过上表可知：

- 簇 1：均值较低，代表需求量较少的配件；标准差较小，需求量波动较小；最小值较低；配件的最小需求量较低；偏度和峰度较接近正态分布，分布相对均匀。

- 簇 2：均值较高，代表需求量较多的配件；标准差较大，需求量波动较大；最大值较高，配件的最大需求量较高；偏度和峰度较大：可能存在较多的离群值或极端需求。
- 簇 3：均值适中，需求量处于中等水平；标准差适中，需求量波动适中；最小值较高，配件的最小需求量较高；偏度和峰度适中，分布相对均匀。
- 簇 4：均值较低，代表需求量较少的配件；标准差适中，需求量波动适中；最小值较低，配件的最小需求量较低；偏度较小，分布相对均匀，没有明显的偏向。
- 簇 5：均值适中，需求量处于中等水平；标准差较小，需求量波动较小；最大值和最小值较为集中，配件的需求量相对稳定；偏度较小，峰度接近正态分布，分布相对均匀。

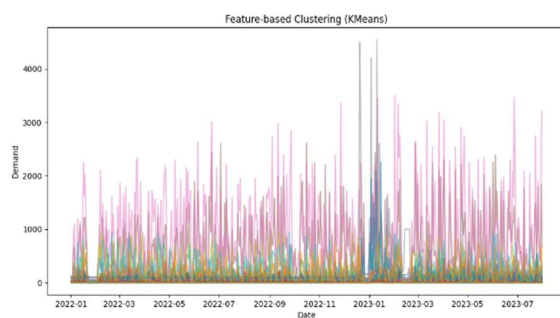


图 10 基于特征的聚类结果图 1

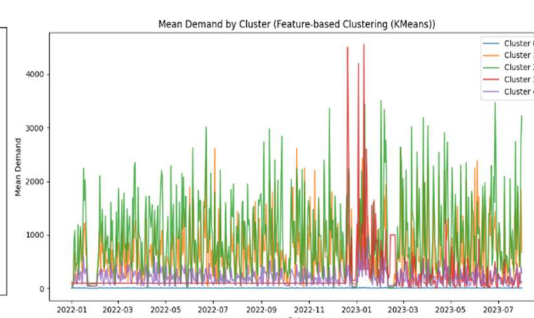


图 11 基于特征的聚类结果图 2

上图可以得出具有相似统计特征的配件需求量时间序列。每条曲线代表一个配件的需求量时间序列，不同颜色和类别代表不同的聚类类别。可以看到一些类别的需求量在某些时间段内具有较高的波动性，而其他类别可能较为平稳。通过观察这些平均曲线，可以进一步了解不同类别的需求模式，例如季节性趋势、周期性波动等。

七、模型的优缺点评价

7.1 模型的优点

(1) XGBoost 模型

对于配件需求量往往受到多种复杂因素的影响，如季节性变化、促销活动、市场趋势等。XGBoost 作为一种基于决策树的集成模型，能够有效捕捉这些非线性关系，提升预测准确性。同时，预测配件需求量需要考虑多种特征，如日期特征（年、月、日、星期几）、历史需求量（滞后特征）等。XGBoost 预测模型可以处理多种类型的特征，并自动进行特征选择，确保最有用的特征被用于模型训练，从而提高预测性能。

(2) K-Means 聚类

能将相似需求模式的配件分组，这样在进行需求预测时，可以针对每个组的时间序列特征使用更为准确的模型。不同组的配件可能具有不同的需求特征，K-Means 聚类可以帮助识别配件需求中的季节性和周期性模式。例如，某些配件在特定时间段内需求量会显著增加，通过聚类分析可以将这些配件归为一类，进而制定针对性的库存和供应策略。

7.2 模型的缺点

(1) XGBoost 模型

需要调节的超参数较多，并且 XGBoost 对输入特征的质量非常敏感，通常需要大量的特征工程来生成有效的特征。在时间序列预测中，这意味着需要生成滞后特征、滚动窗口特征等，这些特征工程过程复杂且耗时。

7.3 模型的优化

XGBoost 模型对输入特征的质量非常敏感，可以利用自动特征工程工具（如 FeatureTools）来自动生成有效的特征从而减少手动特征工程的工作量、特征工程的复杂性和时间成本。采用更高级的特征选择方法（如递归特征消除）简化特征工程过程。在时间序列预测中，Optuna 库进行更高级的超参数调优，找到最优的参数组合。在时间序列预测中，结合深度学习模型（如 LSTM）进行预测，进一步提高模型性能。

八、参考文献

- [1]周明涛,童温亮,章涵,等.基于 GAWOA-XGBoost 改进模型的植被混凝土生境基材配合比研究[J/OL].土木工程学报,1-13[2024-05-20].
- [2]王新,胡安霞.基于 XGBoost 算法的山东省降水量空间插值研究[J].陕西水利,2024,(05):53-55.
- [3]杨骐畅,成子龙,吴奇,等.基于 K-Means 聚类分析的长沙市常规公交与轨道交通竞合关系研究[J].市政技术,2024,42(05):138-144+219.
- [4]梁弘毅,陈继开,刘万里,等.融合 K-means 聚类 and 序列分解的实车锂电池剩余使用寿命预测[J].汽车工程,2024,46(04):634-642.
- [5]王清永,曲伟强.基于线性规划的城市轨道交通运行调度优化算法[J].吉林大学学报(工学版),2023,53(12):3446-3451.
- [6]朱周帆,郝鸿,张立文.基于机器学习与时间序列组合模型的中国汽车市场预测[J].统计与决策,2020,36(08):177-180.
- [7]张镓,霍佳震,胡军,等.基于多元线性回归的上海市生产性服务业和汽车产业发展的关联分析[J].上海管理科学,2012,34(03):49-52.

九、附录

(1) 问题一代码

```
import pandas as pd
import numpy as np
import xgboost as xgb
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import matplotlib.pyplot as plt
from datetime import datetime, timedelta
import os

# Load the data
file_path = 'all.csv' # Path to the CSV file
data = pd.read_csv(file_path, encoding='gbk') # Specify the encoding

# Ensure the '日期' column is datetime type
data['日期'] = pd.to_datetime(data['日期'])

# Create a folder to save the plots
output_folder = 'predictions_plots'
os.makedirs(output_folder, exist_ok=True)

# Initialize an empty DataFrame to store predictions
future_data_all = pd.DataFrame()

# List of unique product codes
product_codes = data['商品编码'].unique()

for product_code in product_codes:
    # Filter data for the current product code
    product_data = data[data['商品编码'] == product_code]

    # Extract date features
    product_data['year'] = product_data['日期'].dt.year
    product_data['month'] = product_data['日期'].dt.month
    product_data['day'] = product_data['日期'].dt.day
    product_data['weekday'] = product_data['日期'].dt.weekday
    product_data['dayofyear'] = product_data['日期'].dt.dayofyear
    product_data['quarter'] = product_data['日期'].dt.quarter

    # Features and target
    X = product_data[['year', 'month', 'day', 'weekday', 'dayofyear',
                      'quarter']]
    y = product_data['需求量']

    # Split the data into training and testing sets
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
```

```

random_state=42)

# Define the parameter grid
param_grid = {
    'max_depth': [3, 5, 7, 9],
    'eta': [0.01, 0.1, 0.2, 0.3],
    'subsample': [0.7, 0.8, 0.9, 1.0],
    'colsample_bytree': [0.7, 0.8, 0.9, 1.0],
    'n_estimators': [100, 200, 300]
}

# Initialize XGBoost model
xgb_model = xgb.XGBRegressor(objective='reg:squarederror')

# Perform grid search
grid_search = GridSearchCV(estimator=xgb_model, param_grid=param_grid,
cv=5, scoring='neg_mean_squared_error', verbose=1)
grid_search.fit(X_train, y_train)

# Get the best model
best_model = grid_search.best_estimator_

# Predict on test data
y_pred = best_model.predict(X_test)

# Calculate evaluation metrics
mse = mean_squared_error(y_test, y_pred)
rmse = mean_squared_error(y_test, y_pred, squared=False)
mae = mean_absolute_error(y_test, y_pred)
mape = np.mean(np.abs((y_test - y_pred) / y_test)) * 100
r2 = r2_score(y_test, y_pred)

print(f'Product Code: {product_code}')
print(f'MSE: {mse}')
print(f'RMSE: {rmse}')
print(f'MAE: {mae}')
print(f'MAPE: {mape}')
print(f'R2 Score: {r2}')

# Plot actual vs predicted
plt.figure(figsize=(14, 7))
plt.plot(product_data['日期'], y, label='Actual Demand')
plt.plot(product_data['日期'][len(X_train):len(X_train) + len(y_pred)],
y_pred, label='Predicted Demand', color='red')
plt.xlabel('Date')
plt.ylabel('Demand')
plt.title(f'Actual vs Predicted Demand for Product Code {product_code}')
plt.legend()

```

```

plt.grid(True)
plt.savefig(os.path.join(output_folder,
f'actual_vs_predicted_{product_code}.png'))
plt.close()

# Predict the next 30 days
future_dates = [product_data['日期'].max() + timedelta(days=i) for i in
range(1, 31)]
future_data = pd.DataFrame({'日期': future_dates})
future_data['year'] = future_data['日期'].dt.year
future_data['month'] = future_data['日期'].dt.month
future_data['day'] = future_data['日期'].dt.day
future_data['weekday'] = future_data['日期'].dt.weekday
future_data['dayofyear'] = future_data['日期'].dt.dayofyear
future_data['quarter'] = future_data['日期'].dt.quarter

X_future = future_data[['year', 'month', 'day', 'weekday', 'dayofyear',
'quarter']]

# Predict future demand
future_preds = best_model.predict(X_future)

# Combine predictions with dates
future_data['预测需求量'] = future_preds
future_data['商品编码'] = product_code

# Append future predictions to the combined DataFrame using pd.concat
future_data_all = pd.concat([future_data_all, future_data],
ignore_index=True)

# Plot future predictions
plt.figure(figsize=(14, 7))
plt.plot(product_data['日期'], y, label='Actual Demand')
plt.plot(future_data['日期'], future_preds, label='Future Predictions',
color='green')
plt.xlabel('Date')
plt.ylabel('Demand')
plt.title(f'Actual Demand and Future Predictions for Product Code
{product_code}')
plt.legend()
plt.grid(True)
plt.savefig(os.path.join(output_folder,
f'future_predictions_{product_code}.png'))
plt.close()

# Save the predictions to an Excel file
future_data_all.to_excel('future_predictions_all_products.xlsx', index=False)

```

```
print("Predictions for all products saved to  
'future_predictions_all_products.xlsx'")
```

(2) 问题 2 代码

```
import pandas as pd
import lightgbm as lgb
from sklearn.model_selection import train_test_split
import numpy as np

# 读取历史配件订单数据
data = pd.read_excel("四舍五入过的.xlsx")

# 转换日期列的数据类型
data['日期'] = pd.to_datetime(data['日期'])

# 将日期拆分为年、月、日
data['年'] = data['日期'].dt.year
data['月'] = data['日期'].dt.month
data['日'] = data['日期'].dt.day

# 定义函数来训练模型并进行分位数预测
def train_and_predict_quantiles(df, quantiles, future_dates):
    # 选择适当的特征和目标
    X = df[['年', '月', '日']]
    y = df['需求量']

    # 将数据划分为训练集和测试集
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

    predictions = {date: {} for date in future_dates}

    for quantile in quantiles:

        model = lgb.LGBMRegressor(objective='quantile', alpha=quantile /
100.0, n_estimators=100, random_state=42)

        # 训练模型
        model.fit(X_train, y_train)

        # 预测未来日期的需求量
        for date in future_dates:
            date_features = np.array([date.year, date.month,
date.day]).reshape(1, -1)
            quantile_pred = model.predict(date_features)[0]
            predictions[date][quantile] = max(0, round(quantile_pred)) # 确保
```

需求量为非负整数

```
        return predictions

# 生成未来 15 天的日期
future_dates = pd.date_range(start='2023-08-01', end='2023-08-15')

# 选择不同配件进行分位数预测
results = []
quantiles = [10, 30, 70, 90]
for sku in data['配件编码'].unique():
    print(f"Predictions for SKU: {sku}")
    sku_data = data[data['配件编码'] == sku]
    preds = train_and_predict_quantiles(sku_data, quantiles, future_dates)
    for date, date_preds in preds.items():
        result = {'配件编码': sku, '日期': date}
        result.update(date_preds)
        results.append(result)

# 将结果转换为 DataFrame 并保存到 CSV 文件
results_df = pd.DataFrame(results)
results_df.to_csv("问题二结果.csv", index=False, encoding='gb2312')

print("预测完成，结果已保存到问题二结果.csv")
```